



AI and AWS

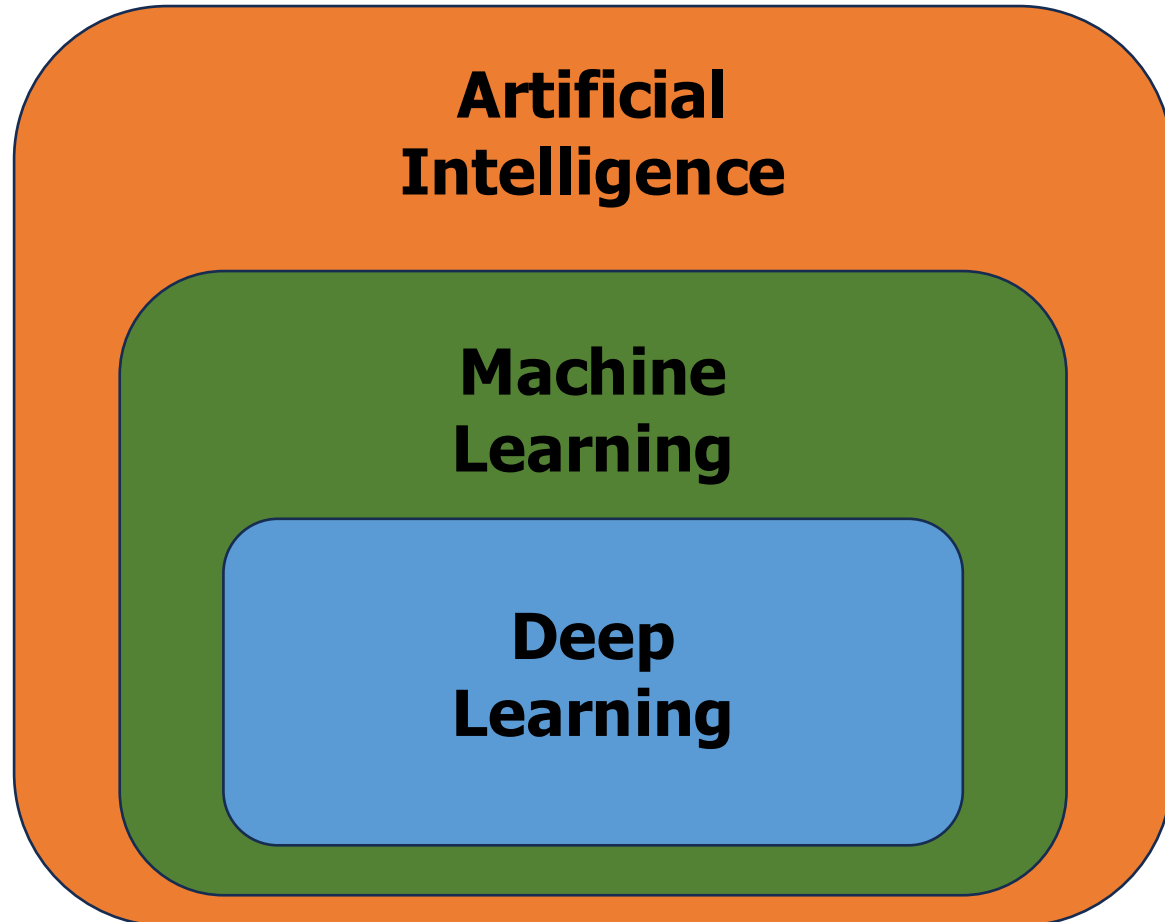
Is AI taking over human jobs? Is it true that AI is becoming the dominant form of intelligence on Earth? Will it take control of the planet?

People with no idea about AI
saying it will take over the world:

My Neural Network:



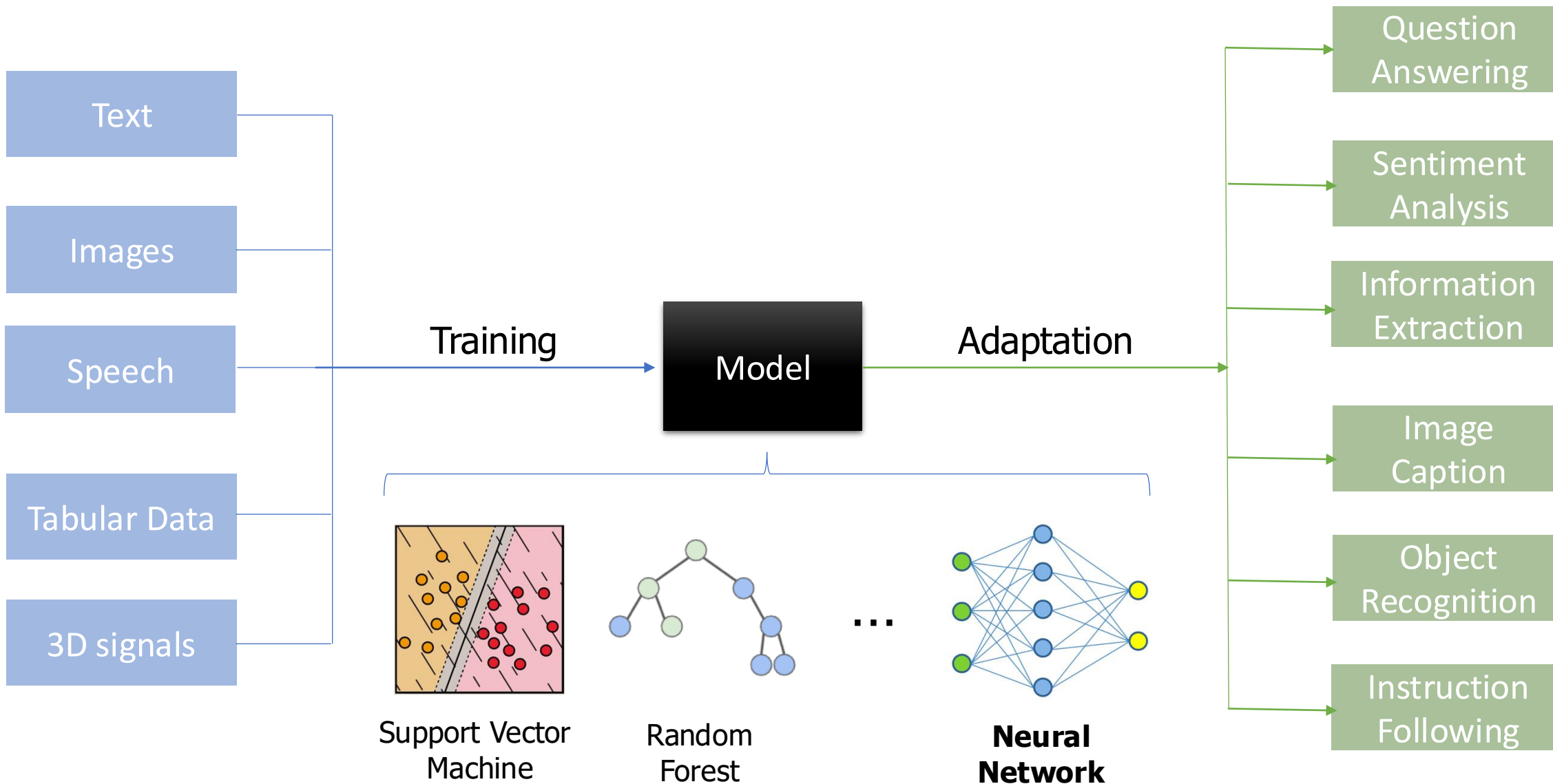
What is AI?



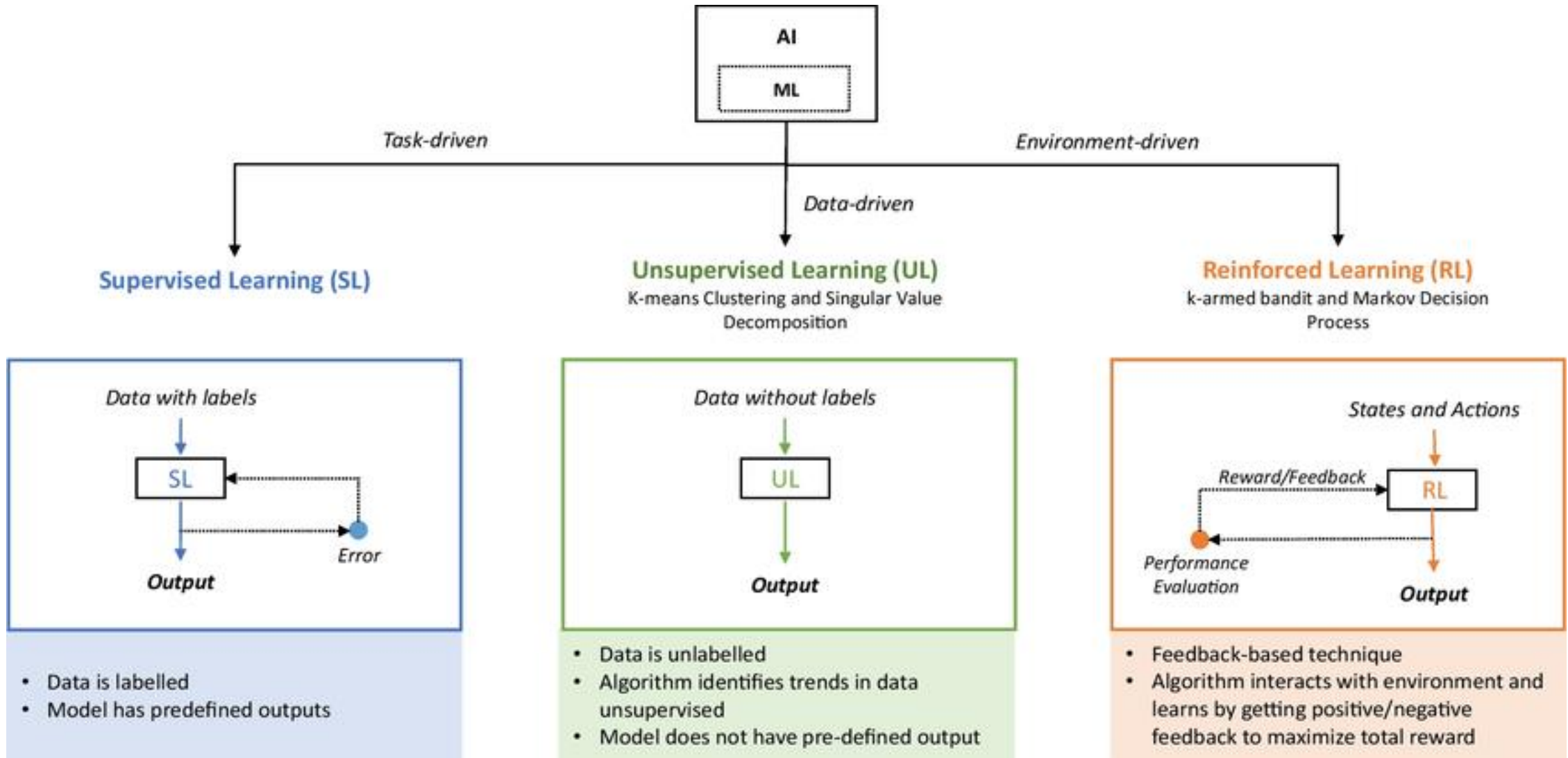
AI: is about making machines act like humans.

Machine Learning: extract knowledge from data and learn from it autonomously.

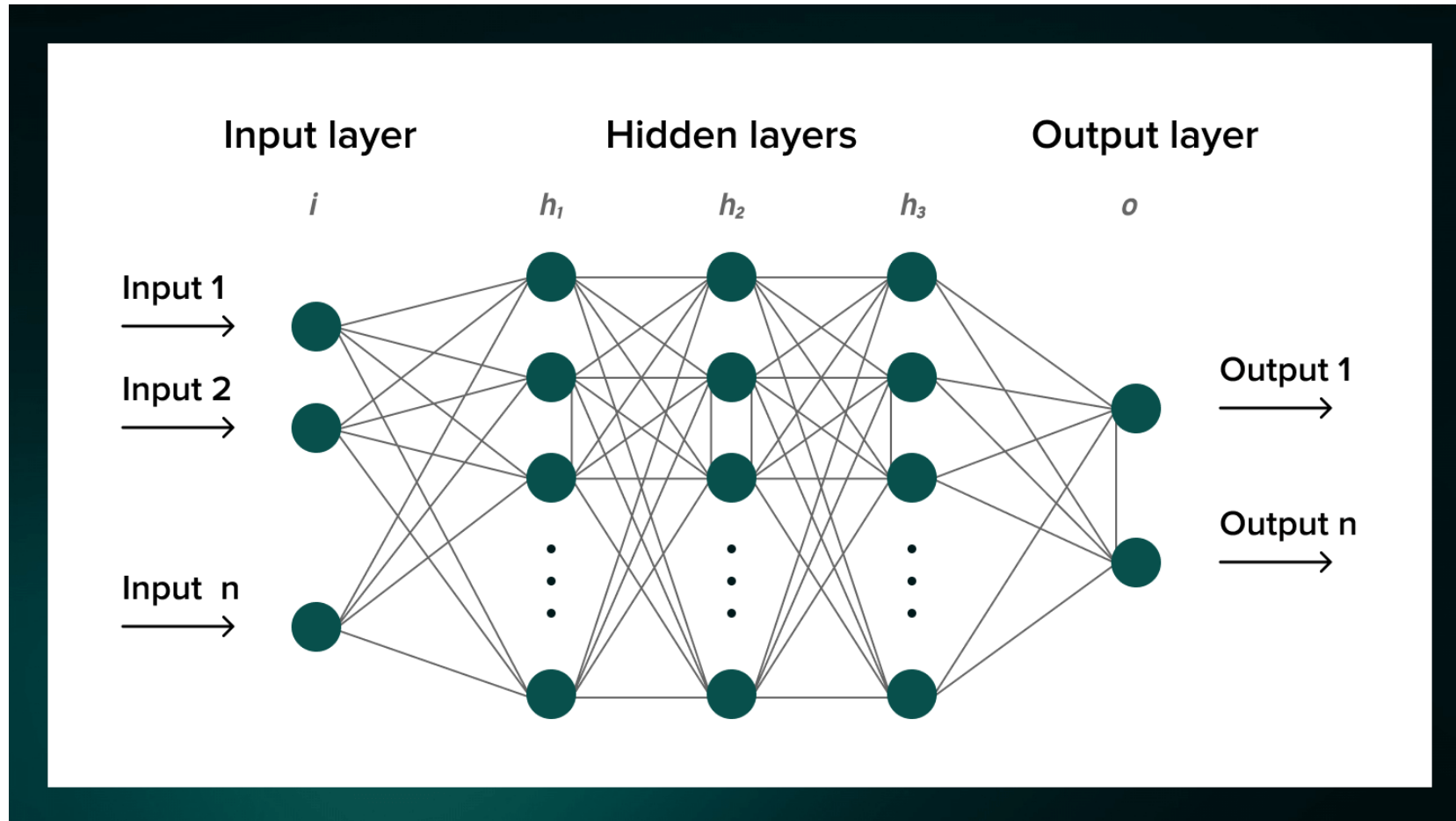
Deep Learning: uses neural network models that mimic biological brain structures and enhance the automation of training AI models.



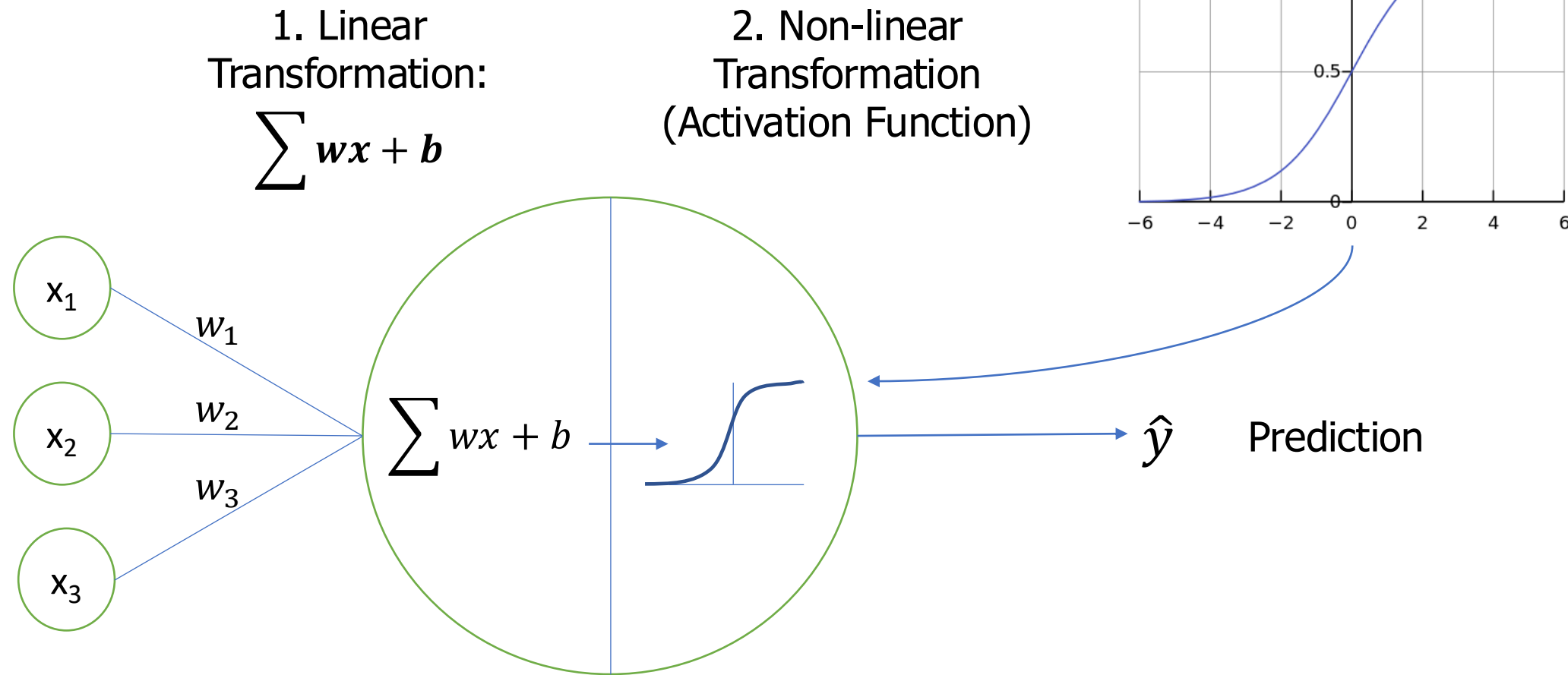
Types of Machine Learning



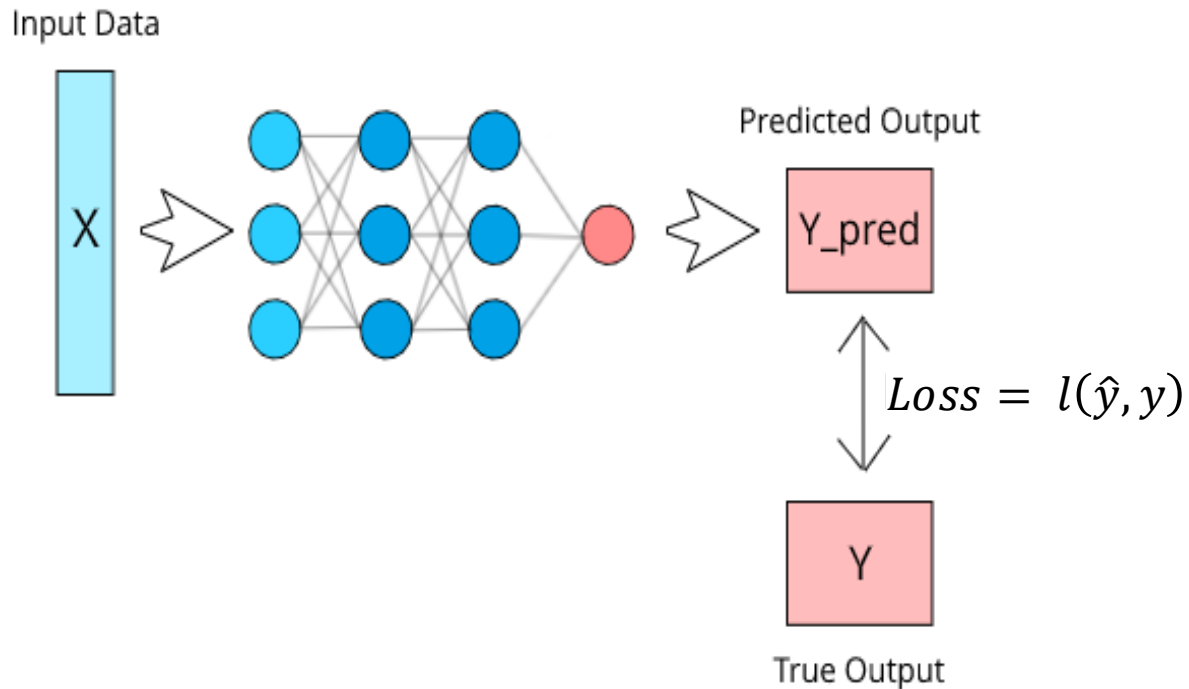
Neural Network Infrastructure



Perceptron (Single node neural network)



Calculating the Loss



- **Loss:** the difference between the prediction and the actual value for the individual data instance.
- Example **Loss Function:** $l(\hat{y}, y) = (\hat{y} - y)^2$
- **Cost:** The aggregate of the losses of all data instances
- Example **Cost Function:** $C(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$
 θ are all the **model parameters: w's and b's**
- **Goal** in training the model: $\min(Cost)$

Classification metrics

- **Accuracy:**
 - Measure the percentage of correctly classified instances.
- **Precision:**
 - Proportion of true positive predictions among all positive predictions.
 - “Out of all positive predictions, how many were correct?”
- **Recall:**
 - Proportion of true positive predictions among all actual positive instances.
 - “Out of all actual positives, how many were correctly identified?”
- **F1 Score:**
 - Harmonic mean of precision and recall, balancing both metrics.

Metrics for Monitoring Model Performance

Actual Outcomes			
		Pos.	Neg.
Predicted Outcomes	Pos.	True Positive (TP)	False Positive (FP)
	Neg.	False Negative (FN)	True Negative (TN)

Actual Outcomes			
		Pos.	Neg.
Predicted Outcomes	Pos.	45	10
	Neg.	5	40

A **confusion matrix** is useful for calculating these metrics.

Example with 100 instances:

- **Accuracy** = $TP + TN / (TP + FP + FN + TN)$
= $85 / 100 = 85\%$
- **Precision** = $TP / (TP + FP)$
= $45 / 55 = 81.82\%$
- **Recall** = $TP / (TP + FN)$
= $45 / 50 = 90\%$
- **F1 Score** = $2 * Precision * Recall / (Precision + Recall)$
= $2 * 0.8182 * 0.90 / (0.90 + 0.8182)$
= 85.71%

Regression and validation metrics

- **Mean Absolute Error (MAE):**

- Commonly used for regression tasks, it measures the average absolute difference between predicted and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE):**

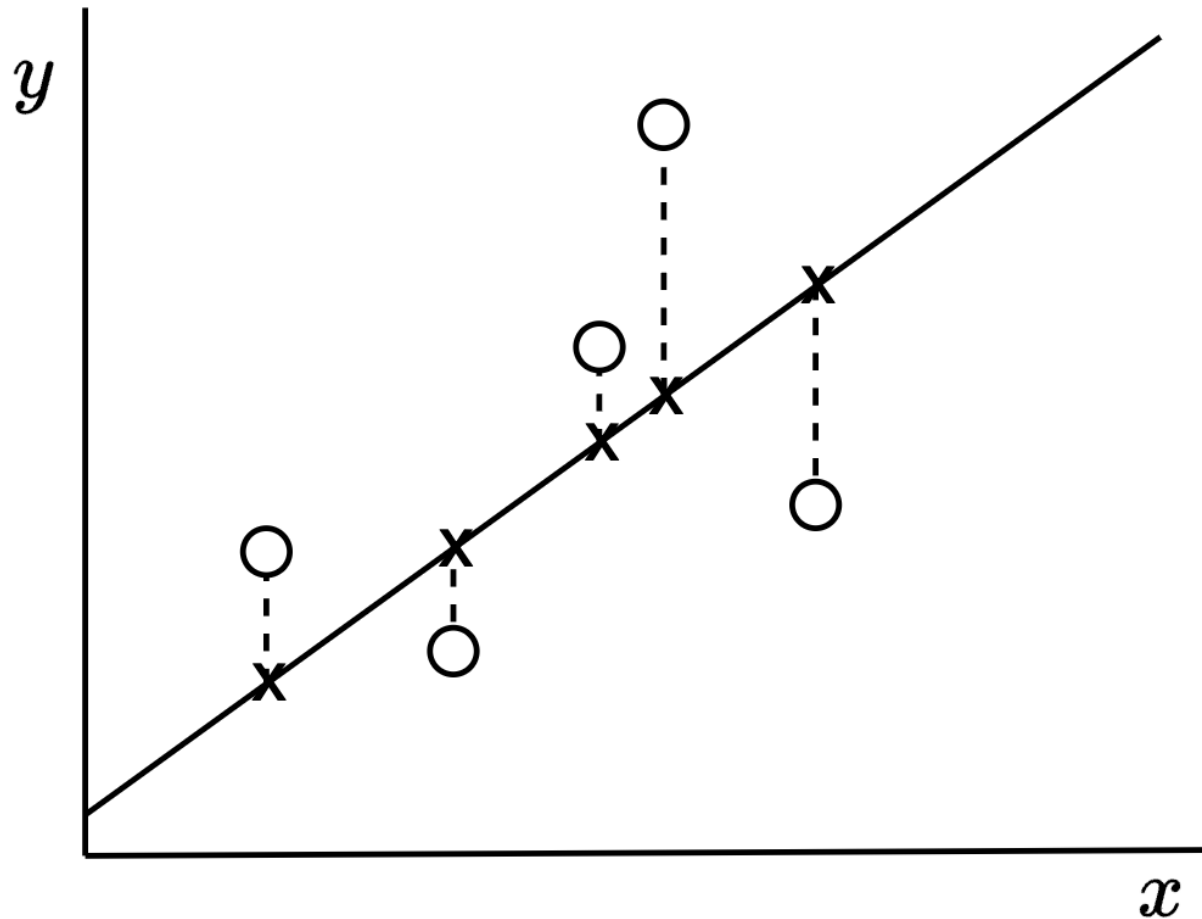
- Another regression metric, measures the average squared difference between predicted and actual values, giving higher weight to larger errors.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Cross-Validation Scores:**

- Assess model performance on multiple subsets of the data to check for overfitting and assess generalization.

Metrics for Monitoring Model Performance



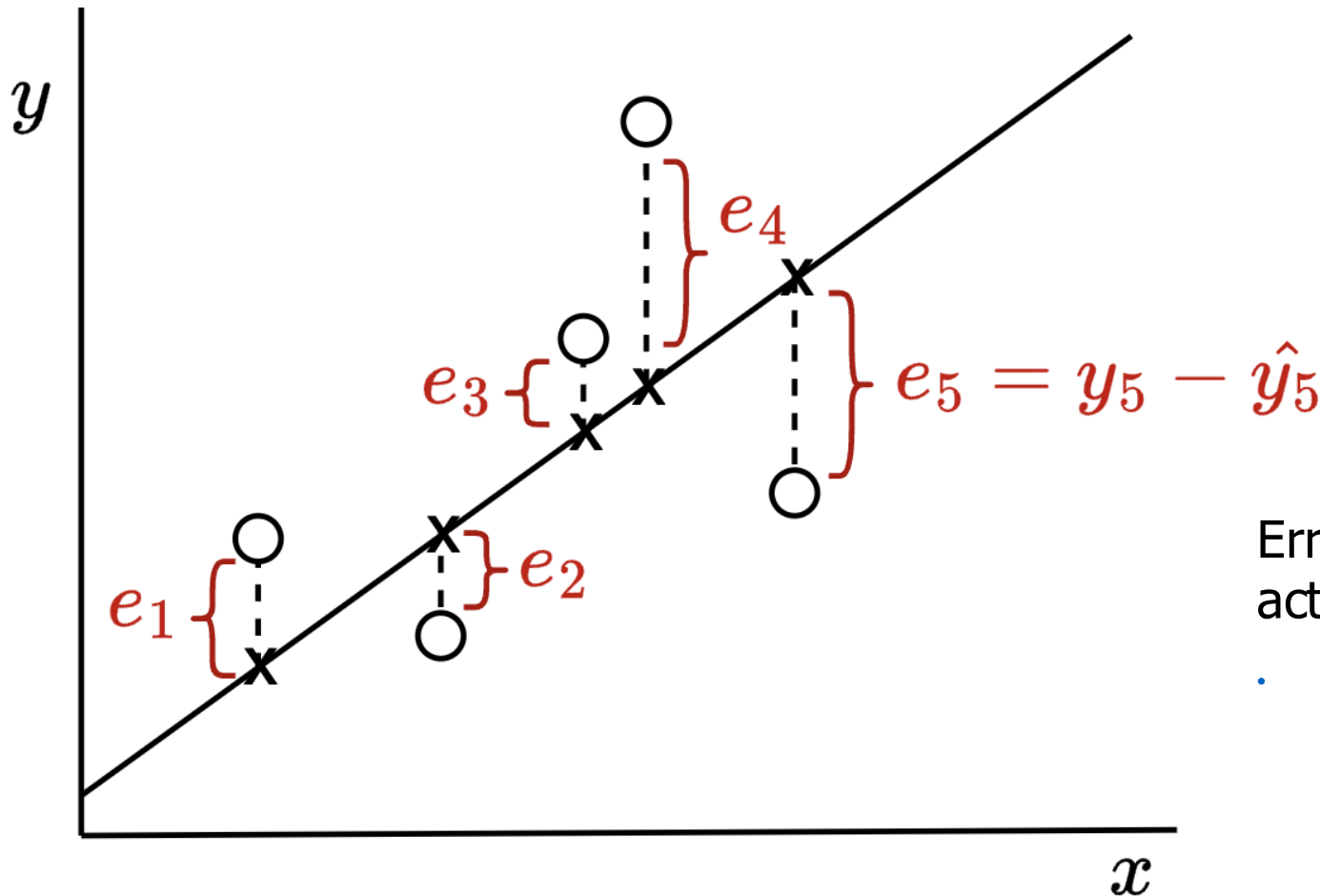
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Example regression problem:

- Dots are actual data points.
- Line is the learned function (“line of best fit”).
- X are the intercepts.

Metrics for Monitoring Model Performance



$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Errors **e** are the differences between the actual **y** and the predicted **$\hat{y}$**

- You can see how the MSE penalises larger errors.

The machine learning lifecycle

1. Prepare the environment
2. Define the task and prepare the data
3. Select the model
4. Preprocess the data
5. Train and configure hyperparameters
6. Evaluate the model
7. Deploy the model and run inference

Ownership of the ML lifecycle

- **Demo 1: consume a trained capability**
 - Rekognition: AWS owns the model lifecycle; we supply the input and use the result.
- **Demo 2: configure a general-purpose model**
 - Bedrock: we select a model, design prompts, and evaluate its responses.
- **Demo 3: build and operate a custom model**
 - SageMaker AI: we take responsibility for data, training, evaluation, and deployment.

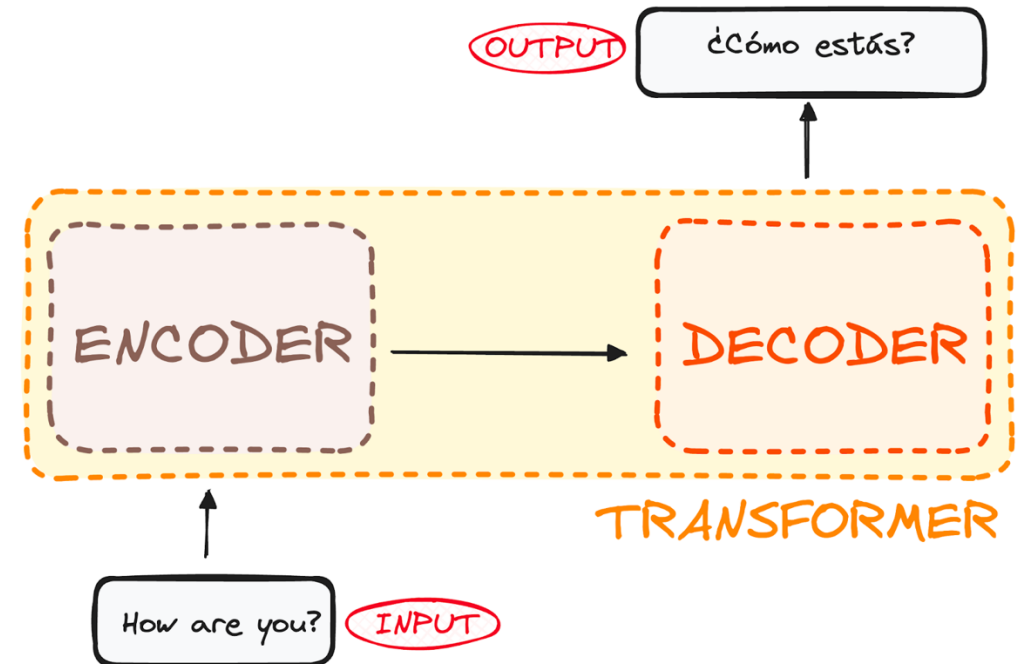
Across the demos, more of the ML lifecycle becomes our responsibility.

Demo 1: Consume a managed AI service

- **Architecture: S3, Lambda, Rekognition, DynamoDB, CloudWatch.**
- Demo action: upload an image to S3 and inspect the labels stored by the triggered workflow.
- We own: the event flow, application logic, data storage, and interpretation of the result.
- AWS manages: model development, training, tuning, hosting, and scaling.
- Lifecycle view: we begin near deployment and inference; most earlier stages are hidden.
- **Key idea: We are consuming a trained capability, not building a model.**

Foundation models and prompting

- Transformers use attention to model relationships across a sequence.
- Foundation models train on broad datasets and can support many downstream tasks.
- Applications usually begin by selecting a model and designing prompts.
- **Prompting changes the model's instructions and context; it does not change the model weights.**



Demo 2: Configure a foundation model

- **Application path: client request, API Gateway, Lambda, Amazon Bedrock.**
- We own: model choice, prompt design, application logic, and response evaluation.
- AWS manages: pre-training, model hosting, infrastructure, and scaling.
- Lifecycle view: model selection and evaluation become more visible, but we still do not train the model.
- **Key idea: A foundation model is flexible, but it remains a managed component in this demo.**

Demo 3: Build and operate a custom model

- **Workflow: S3, SageMaker hyperparameter tuning, model artifact, endpoint.**
- Store prepared training and validation data in S3.
- Launch several training jobs across a hyperparameter search space.
- Compare jobs using one objective metric and select the best model artifact.
- Deploy the selected model to an endpoint and invoke it with new data.
- **Lifecycle view: We now own the decisions across the complete ML lifecycle.**

1. Preparing your Environment

- **Hardware and Software Requirements:** Ensure you have access to a suitable environment for training.
 - AWS EC2 Instances, S3 buckets, EBS, etc., and associated policies.
 - Amazon Elastic Inference: Elastic Inference allows you to attach low-cost GPU-powered inference acceleration to your EC2 instances.
 - Amazon CloudWatch: CloudWatch provides monitoring and logging
- **Install Required Libraries:** Use Pip to install the necessary Python libraries. Common libraries:
 - NumPy and Pandas (data manipulation)
 - Matplotlib and Seaborn (visualisation)
 - Scikit-learn (for ML) TensorFlow or PyTorch (for deep learning).

2. Task Definition and Data Preparation

- **Task Definition:**

- What is the problem to solve, and what business logic do we need to do so?
- Data Domain: image, text, audio, tabular?
- Task: classification, regression, generative outcomes?

- **Data Collection:**

- Identify and gather relevant data sources (e.g., databases, APIs, web scraping).
- Ensure the data is clean, accurate, and representative of the problem domain.
- (Try to) collect sufficient data to train robust models, considering the complexity of the task.

3. Model Selection

- **What model are we going to use?**
- Certain ML/AI models work best in specific contexts (below is a generalisation):
 - Convolution Neural Networks (CNNs) are ideal for images.
 - Sequence models (LSTM, RNN, Transformers) are ideal for text.
 - Tree-based models (Random Forests, Boosting models) are suitable for tabular, non-structured data.
- **Evaluate Costs Involved**
 - Select the size of the model that is appropriate for the scale of your data, and your budget.
 - More data = larger models = higher costs.

4. Data Preprocessing

- **Steps needed to prepare data for ML**

- Handling missing values via imputation (replace missing values with substitutes), removing the datapoint itself, etc.
- Ensure features are on a comparable scale, crucial for many machine learning algorithms.
 - Standardization: Centres data by removing the mean and scales it by dividing it by the standard deviation.
 - Normalization: Rescales data to a fixed range, typically $[0, 1]$, which is helpful for algorithms like neural networks that are sensitive to the scale of input data.

- **Some preprocessing steps are data format-specific:**

- Text: tokenisation, stop-word removal, stemming/lemmatization, etc.
- Tabular Data: encoding categorical values, etc.

5. Training and Hyperparameterisation

- Model ingests your data and extracts patterns/knowledge from it, and updates these beliefs iteratively as more data is feed through.
- Important considerations:
 - **Loss Function:** Define a loss function that calculates the error between model predictions and ground truth.
 - **Optimizer:** Choose an optimizer such as Adam or RMSprop to update the model's weights during training.
 - **Training Loop:** Implement a training loop that iterates through your dataset.
 - **Validation:** Periodically evaluate your model on a validation dataset to check its performance during training.

5. Training and Hyperparameterisation

- **Training:** model ingests your data and extracts patterns/knowledge from it, and updates these beliefs iteratively as more data is feed through.
- **Hyperparameters:** settings that control the learning process.
 - The hyperparameters and their values depend on your specific task and dataset.

6. Evaluation

- **Test Data:** evaluate your fine-tuned model on a separate test dataset.
 - **Metrics:** Calculate and report relevant evaluation metrics for your specific task: Accuracy, Precision, Recall, F1, AUC, etc.
- **Monitoring:** Continuously monitor your training and evaluating process.
- **Debugging:** If your model isn't performing as expected, check your data, preprocessing, and model architecture for issues.

7. Deployment and Inference

- **Model Deployment**

- Make a trained model available for use in a production environment.
- Deploy models to endpoints for real-time or batch predictions. Users can then access the model via the endpoint(s).

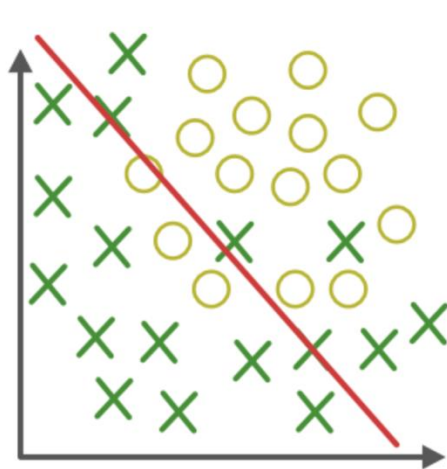
- **Inference**

- Applying a trained machine learning model to new, unseen data to generate predictions or decisions.

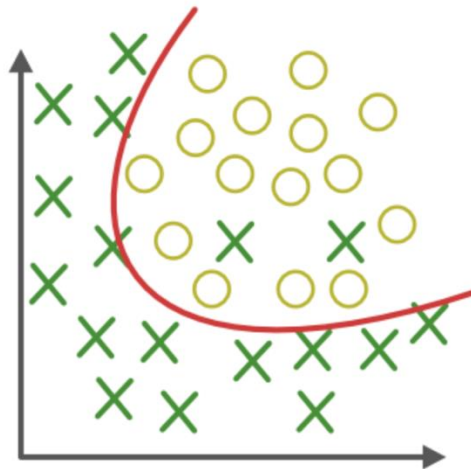
Training risk: overfitting

- **Overfitting**

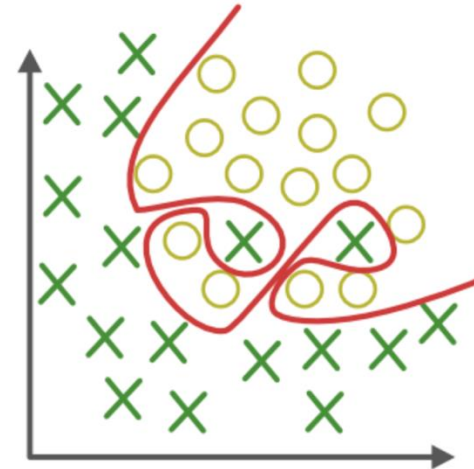
- The model cannot generalize and fits too closely to the training dataset, and cannot predict new data well; it performs well on training data but poorly on unseen data.
- Often due to noisy data, using too small of a training dataset, or using too complex of a model relative to the data.



Underfitting



Just Right

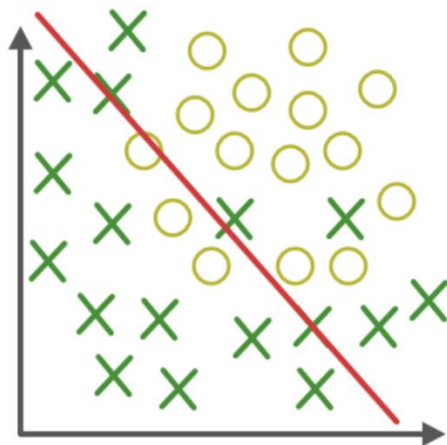


Overfitting

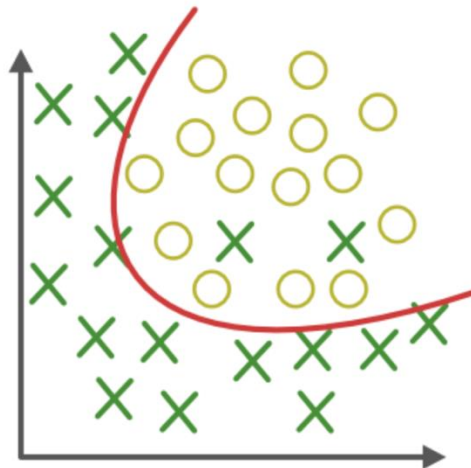
Training risk: underfitting

- **Underfitting**

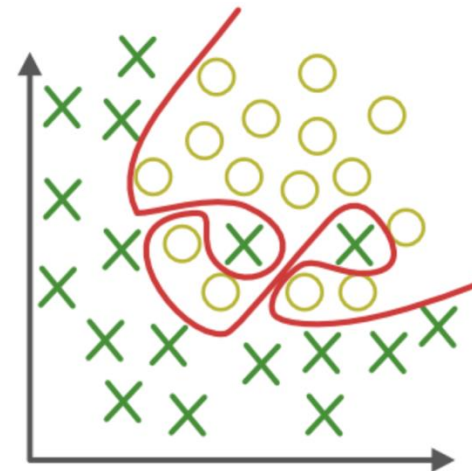
- Model fails to capture patterns and does not learn the underlying trend in the data, leading to poor performance on both training and test data.
- Often due to using too few parameters, or overly simplistic assumptions, or not training a sufficiently complex model for long enough.



Underfitting



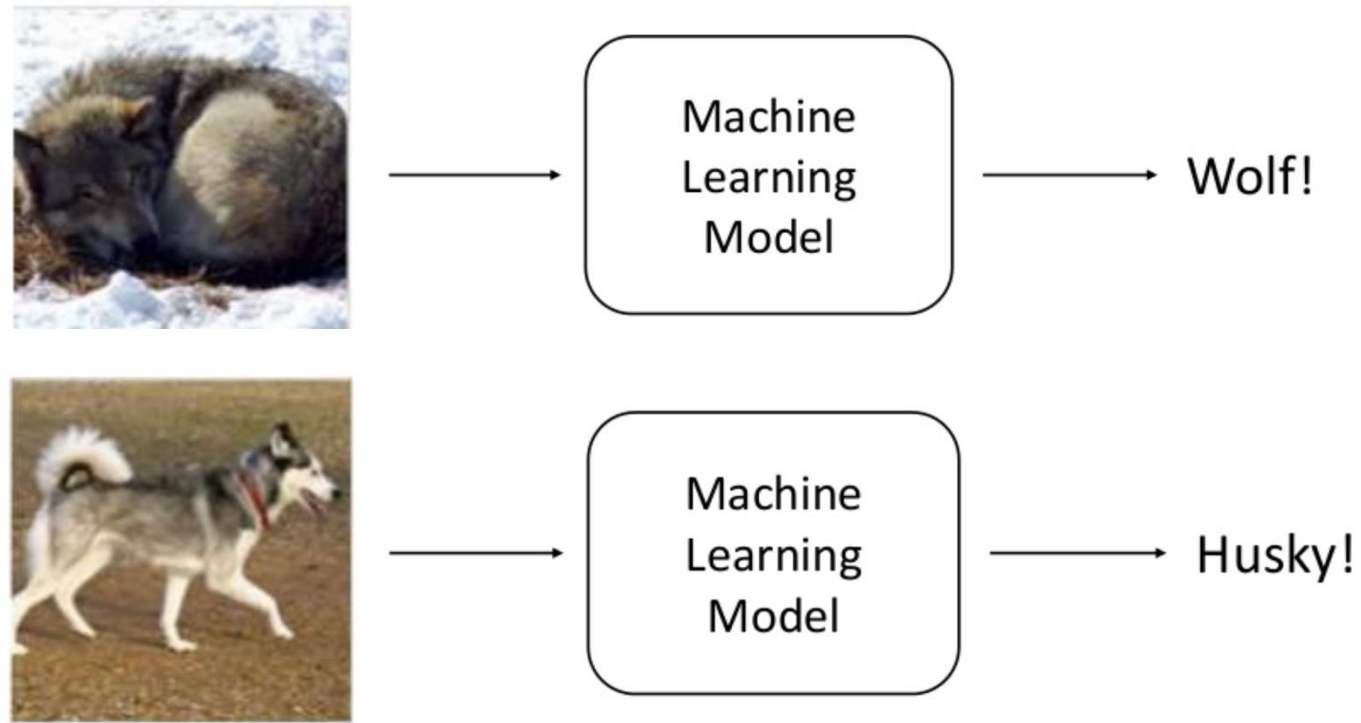
Just Right



Overfitting

Evaluation risk: misleading validation

- **Having Too Much Confidence in Model Outcomes**
 - Example: Classifier to recognize images as either Wolf or Husky



Evaluation risk: misleading validation

- **Having Too Much Confidence in Model Outcomes**

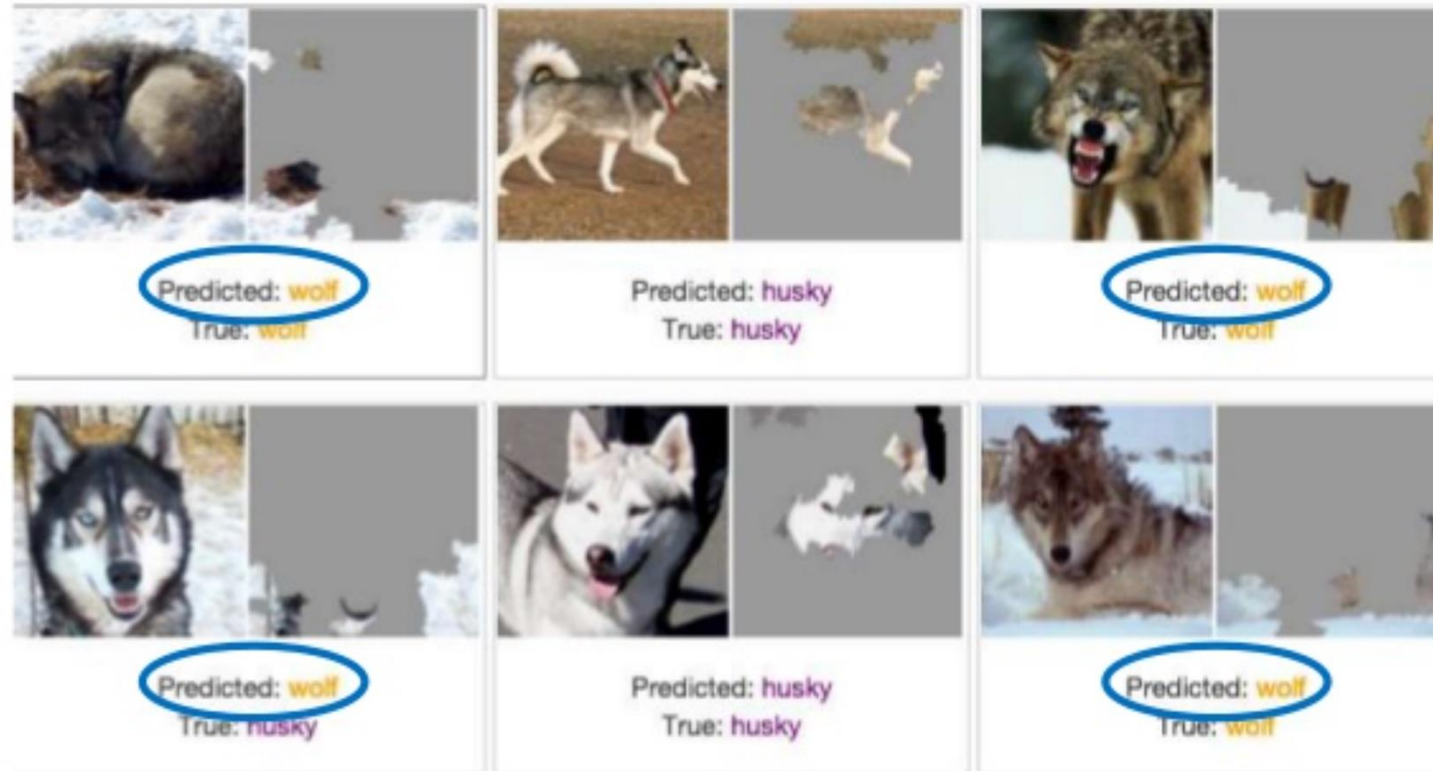
- Predictions indicate strong performance!
- Does this mean we have a good model?

Only 1 mistake!



Evaluation risk: misleading validation

- **Having Too Much Confidence in Model Outcomes**
 - Turns out, the model was recognising snow as the distinguishing feature for wolf!



Further Interest Topics

- **Model Interpretability**

- AI models are “black boxes”, but large research efforts are put into model explainability and interpretability.

- **Model Hosting and Scalability**

- Cost effective hosting strategies as models scale and use-cases grow.
- Multi-model deployment, AWS AutoScaling, AWS DeepSpeed Inference.

- **Beyond AWS**

- AI/ML services on other cloud platforms like Google Cloud AI, Microsoft Azure AI, and even GitHub.

Thanks!

- Questions and conversations: larry.huynh@uwa.edu.au